TRAINEE HANDBOOK · 12 WEEKS · 36 CONTACT HOURS

Agentic Software Engineering

Systems analysis and software development in the agentic era: framing problems, writing executable specifications, directing AI coding agents through a harness, and owning what comes back.

01 Systems analysis: what should exist, and how will we know it works?

02 Security: how does it fail when the attacker, the data or the agent is hostile?

03 Agentic practice: reliable, reviewable work from a harness at acceptable cost



SPECIFY · VERIFY · OWN

Alvin O. Forteta

Version 3.0 · 10 November 2026 · Trainee edition

Case study: your university's ERP, built module by module on GitHub

00 CONTENTS

Contents

Parts 01–08 are the course design. Part 09 holds one session plan per week, matching that week's slide deck.

01 Course overview

02 Course philosophy and design

03 Toolchain and environment

04 Weekly schedule

05 Capstone project

06 Assessment and grading

07 Agentic best-practices playbook

08 Reading list and resources

09 Weekly session plans

01 PART 01 OF 09

Course overview

Agentic Software Engineering is a 12-week, studio-format course in which students do not write most of the code. They frame problems, write executable specifications, direct AI coding agents through a harness, verify and harden what comes back, and are held accountable for the result. The course assumes that by the time these students graduate, the unit of engineering work is no longer the line of code but the specification, the verification, and the decision about how much autonomy to grant.

Working title for the catalogue: *Systems Analysis and Software Engineering in the Agentic Era*

Level: final-year undergraduate or first-year postgraduate in Information Systems, Computer Science or Software Engineering. A graduate and an executive variant are sketched in the last section.

Contact time: 3 hours per week (1 hour seminar, 2 hours studio) plus roughly 6 hours of self-directed agent work. 12 weeks, 36 contact hours.

Prerequisites

- One programming course (any language) and comfort with Git and the command line
- An introductory systems analysis or software engineering course
- A personal GitHub account, created and secured (two-factor authentication, SSH key) before week 1 using the setup guide issued at enrolment; every trainee works from their own account for the whole course
- No prior AI/ML background required; the course teaches what a harness is from the inside

Three strands, braided through every week

Strand	What it answers	Where it lands in the capstone
Systems analysis	What should exist, for whom, and how will we know it works?	The executable specification the agents build against
Security	How does this system fail when the attacker, the data or the agent itself is hostile?	Threat model, hardened harness, red-team report
Agentic practice	How do you get reliable, reviewable work out of an agent harness at acceptable cost?	Harness configuration, transcripts, eval suite

Learning outcomes. By the end of the course a student can:

1. Frame an ill-structured organisational problem and turn it into an executable specification (domain model, acceptance tests, architecture decision records) that an agent can build against without further clarification.
2. Explain how an agentic harness works internally (the loop, context window, tool calls, memory, sub-agents) and build a minimal one.
3. Configure a production harness for a repository: context files, skills, hooks, permissions, MCP tools, sub-agents, and evals.
4. Decompose a feature into agent-sized tasks, run them in parallel where safe, and review the resulting pull requests with evidence rather than trust.

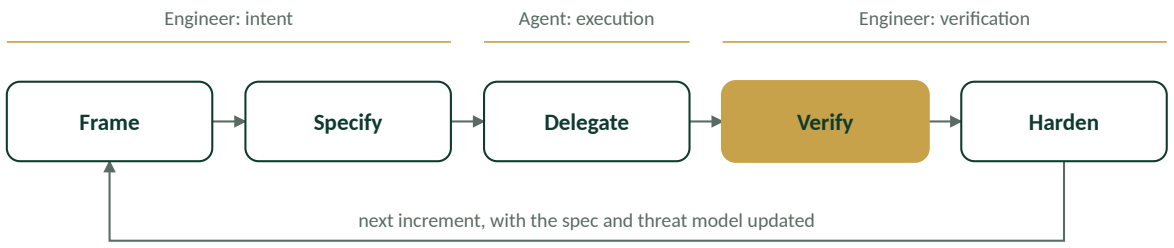
5. Threat-model an agentic system against the OWASP Top 10 for Agentic Applications and the OWASP Top 10 for LLM Applications, then demonstrate both an attack and its mitigation.
6. Decide, and justify in writing, how much autonomy to grant an agent for a given task, and design the human gates around it.
7. Operate an agent-built system: observe it, measure its cost, respond to an incident, and keep it agent-readable as it evolves.
8. Defend an agent-built system in an adversarial viva, including the transcripts of how it was built.

02 PART 02 OF 09

Course philosophy and design

The central claim: in agentic engineering the engineer owns *intent* and *verification*; the agent owns *execution*. Everything in the course is organised around making students excellent at the two ends of that loop and sceptical about the middle.

The engineer owns intent and verification; the agent owns execution



The agentic engineering loop · five steps, three roles.

Framing and specification come first, delegation is the only step the agent owns, and verification (the accented step) is where marks are earned; hardening feeds the next increment.

Five design decisions that make this course different from a tools course

1. **The course is itself agent-native.** Every student works in a Git repository that carries an `AGENTS.md`, a skills folder, hooks and a permissions policy from week 1. Assignments are submitted as pull requests, and the agent transcripts that produced them are part of the submission. The instructor uses agents to pre-review submissions and to generate adversarial test cases; students know this and design for it.
2. **Specification before code, always.** Students may not ask an agent to build anything in weeks 3 to 5 until the acceptance tests exist. The rule is relaxed later only when they can justify it in their engineering journal. This is where the systems analysis strand earns its place: the quality of the spec is the ceiling on the quality of the build.
3. **Students build a harness before they use one in anger.** In week 2 each student writes a minimal agent loop (a model, a tool registry, a context budget, a stop condition) in under 300 lines. Nothing in the rest of the course is magic after that: hooks, sub-agents, context compaction and permission gates are all recognisable as things they have already built badly once.
4. **Security is a strand, not a week.** The OWASP Agentic Top 10 is introduced in week 1 as a vocabulary and revisited every week as the threat model of whatever was just built. Weeks 7 and 8 go deep; week 8 is a live red-team against classmates' capstones.
5. **Autonomy is a design variable.** Every task in the course is tagged with an autonomy level (L0 suggest, L1 draft for review, L2 act then report, L3 act unattended within gates). Students must choose the level, justify it, and live with the consequences. This is the single most transferable judgement the course teaches.

The weekly rhythm

- *Seminar (1 h):* one idea, one reading, one worked example from a real transcript, good or bad.
- *Studio (2 h):* instructor and agents in the room; students work on the week's lab in pairs, one driving the harness and one reviewing, swapping at the half.
- *Self-directed (6 h):* the lab finished, the engineering journal written, and the capstone advanced by one milestone.

The engineering journal. A short weekly entry in the repository answering three questions: what did I delegate, what did I verify and how, and what would I not delegate again. It is graded for honesty and specificity, not for success.

Why this matters in Nigeria specifically. Agentic tools compress the gap between a small team and a large one more than any previous technology. A three-person team at a Nigerian university or startup can now specify, build and operate systems that previously needed thirty people. The constraint shifts from headcount to judgement, specification quality and security discipline, which is exactly what this course trains. Cost and connectivity are treated as first-class engineering constraints rather than footnotes (see the toolchain section).

03 PART 03 OF 09

Toolchain and environment

The course is harness-agnostic by design: students learn the concepts on one primary harness and are required to port their configuration to a second one in week 9, so they leave knowing what is standard and what is vendor-specific. Pick one primary harness per cohort and keep it for the twelve weeks.

Layer	What students use	Why this choice
Primary harness	A terminal-based coding agent (Claude Code or equivalent)	Transcripts are plain text, hooks and permissions are explicit, sub-agents and worktrees are native
Secondary harness (week 9 port)	An IDE-integrated or open-source agent	Forces students to separate the standard (`AGENTS.md` , MCP) from the proprietary
Cross-tool context	`AGENTS.md` at the repo root, kept under 150 lines, with tool-specific addenda files	Now the de facto cross-tool standard, read by the major coding agents
Tool protocol	MCP servers, one per external system, with schemas and one-line intent statements	The default integration protocol for agents; also the main supply-chain attack surface, which the security strand exploits
Sandbox	Containers per student with a network allow-list; a disposable cloud VM for the red-team week	Agents must be able to break things without breaking the course
Repository platform	GitHub, required: each trainee's own account, a course organisation, pull requests with required reviews, branch protection, GitHub Actions for CI and for the scheduled agent, Codespaces as an optional cloud workspace	PRs plus transcripts are the unit of submission
Observability	Transcript logging, token and cost accounting per task, a simple eval runner	Students must see what their agents cost and where they fail
Models	One frontier hosted model for the harness; one small open-weight model run locally for the week 2 "build a harness" lab and as a cost fallback	Teaches that model choice is an engineering decision, not a default

Cost model. Budget roughly 15 to 30 USD of frontier-model usage per student per week at current prices, front-loaded in weeks 5 to 6 and 10 to 12. Three cost controls are built into the course rather than left to chance:

1. Every task has a token budget declared up front in the task file; exceeding it without a journal note costs marks.
2. Cheap-model-first routing: plan and review with the small model, build with the frontier model, and record the difference in outcome. This is itself a graded exercise in week 10.
3. Context hygiene is assessed: bloated `AGENTS.md` files, unbounded tool lists and un-compacted sessions are visible in the transcripts and are marked down.

Connectivity. All labs are designed to complete on a laptop with intermittent connectivity: the harness is local, the repository syncs when online, and the local model covers the gaps. The cloud VM is needed only in weeks 8 and 12.

GitHub in the course. GitHub is the system of record for everything a trainee produces and the place where agents and humans meet. Trainees are required to set up and own their GitHub accounts; the course never issues shared or institutional accounts, because the portfolio of PRs, transcripts and journal entries is theirs to take with them. Every week adds one GitHub skill, used immediately in that week's lab and assignment.

Week	GitHub skill	Used for
1	Account, 2FA, SSH key, course organisation, fork, first PR, issue and PR templates	Submitting every lab from here on; process visible by default
2	Branch per task, draft pull requests, commit hygiene	The mini-harness and its CI test
3	Module folder, team ownership, project board, issues as the elicitation log	Your team module and its charter
4	Branch protection, required reviews, CODEOWNERS on your module and its spec	No module or spec changes without its owners' review
5	GitHub Actions CI, worktrees mapped to branches, draft PRs for agent work	Landing agent-built features through the same gate as human work
6	PR review tooling, required checks, test reports as artifacts	Agent PRs judged on evidence, not claims
7	Dependabot, secret scanning, dependency review, security advisories, SECURITY.md	Supply-chain and secrets controls in the ERP threat model
8	Fine-grained personal access tokens, least privilege, revoking a leaked token	Red-team week: what an over-privileged token lets an agent do
9	Repository hygiene, templates, Codespaces as a portable environment	An ERP any agent or person can open and run
10	Scheduled Actions, environments with required reviewers, workflow cancellation	Your module's L3 agent with a human gate and a kill switch
11	Repository rulesets, audit log, licence, SECURITY.md and GOVERNANCE.md	Governance evidence for the viva
12	Release tags, GitHub Releases, the repository as a portfolio	The final ERP release and your portfolio

What lives on GitHub. The course runs in one GitHub organisation, built by the setup script in the instructor kit. Trainees get read access to the course repositories and write access to the ERP through the `trainees` team; each module team owns its folder through CODEOWNERS.

Repository	Visibility	What it is for
`.github`	Public	Organisation profile, default templates, and the trainee setup guide (`SETUP.md`)

Repository	Visibility	What it is for
`course-tracker`	Org only	Assignment issues A1–A12, one submission-row issue per trainee, weekly milestones, Projects board
`legacy-registry`	Org only	The old ERP's Registry app: deliberately flawed, with seeded bugs, vulnerabilities, smells and planted agent-injection traps; forked in week 1, threat-modelled in week 7, refactored in week 9
`mini-harness-starter`	Org only (template)	Week 2 skeleton and the 11 tests the harness must pass
`<handle>-labs`	Org only	Each trainee's individual work (weeks 1–2)
`university-erp`	Org only (trainees write; CODEOWNERS gate merges)	The shared ERP with all the course guard rails: `AGENTS.md`, harness permissions and hooks, reviewer sub-agent, skills, PR template, CI, evals, threat-model and governance templates, the gated scheduled agent; the core, the class Course Registration module, and one folder per team module

04 PART 04 OF 09

Weekly schedule

The twelve weeks run in four blocks: **Foundations** (1–2), **Specify** (3–5), **Verify and secure** (6–8), **Operate and own** (9–12). Each week names its strand emphasis, the seminar idea, the studio lab, and the artefact that lands in the repository.

Wk	Block / strand	Seminar idea	Studio lab	Artefact in the repo
1	Foundations · all three	The agentic turn: from writing code to owning intent and verification. Autonomy levels L0–L3. OWASP Agentic Top 10 as vocabulary	Set up GitHub (own account, 2FA, SSH key, join the course organisation). Fork the old ERP's Registry app (`legacy-registry`) and fix one bug at L1 with a PR; write `AGENTS.md` v1. Clone `university-erp` , seed it, and propose your university's structure in `institution.yaml`	Bug-fix PR with transcript; `institution.yaml` PR; journal entry 1
2	Foundations · agentic	Inside the harness: the loop, context window, tool calls, memory, compaction, sub-agents, stop conditions	Build a minimal agent loop (< 300 lines) with a tool registry and a token budget, against the local model. Make it fail, then fix the failure	`mini-harness/` with three tools and a test; journal entry 2
3	Specify · systems analysis	Problem framing for ill-structured situations: rich pictures, stakeholders, CATWOE-style root definitions; agents as elicitation partners and synthesisers, never as the client	Class: interview the Deputy Registrar agent about Course Registration. Teams: choose an ERP module, interview its stakeholder agent (and the real office if possible), and get a problem statement signed	Module chosen and created; signed problem statement; stakeholder map
4	Specify · systems analysis	Executable specification: domain model, acceptance tests as contracts, architecture decision records as agent context. Spec-driven development	Write the capstone's domain model, ten acceptance tests and three ADRs. No build allowed yet	`spec/` folder; failing acceptance suite; ADR-001..003
5	Specify · agentic	Agentic workflow patterns: plan → implement → verify loops, task decomposition, parallel worktrees, sub-agents, skills, hooks, MCP tools. Context engineering as a discipline	Decompose the first capstone feature into agent-sized tasks; run two in parallel worktrees with a reviewer sub-agent; land them through GitHub Actions CI and a protected main branch	Feature 1 merged; task files; hooks and a first skill

Wk	Block / strand	Seminar idea	Studio lab	Artefact in the repo
6	Verify · agentic	Verification without trust: reviewing agent PRs with evidence, mutation testing, property-based tests, evals for agent workflows, and the "it works on the transcript" fallacy	Build an eval suite for your own harness configuration; break your agent with five adversarial tasks and measure pass rate before and after fixes	`evals/` with results table; journal entry 6
7	Secure · security	Threat modelling agentic systems: STRIDE extended to agents, the OWASP Agentic Top 10 (goal hijack, tool misuse, identity, supply chain, code execution, memory, inter-agent, cascading failures, trust exploitation, rogue agents) and the LLM Top 10	Threat-model the capstone and the harness that builds it; map every finding to a control; write the security section of the spec	`THREATMODEL.md`; controls backlog; hardened permissions policy
8	Secure · security	Attacks in practice: indirect prompt injection via files, issues and web pages; poisoned MCP servers and skills; over-privileged tools; secret leakage; memory poisoning	Red-team / blue-team: teams attack each other's capstones on disposable VMs for 60 minutes, then defend. Every successful attack must be reproduced and patched	Red-team report; patches; journal entry 8 (the honest one)
9	Operate · systems analysis + agentic	Agent-native architecture: designing systems that agents build and operate — legible repos, narrow tool APIs, typed boundaries, ADRs as memory. Portability across harnesses	Integrate modules through the published events and test one cross-module journey; refactor for agent-readiness; port the harness configuration to the secondary harness	Integration test; agent-readiness score before/after; second-harness config
10	Operate · agentic	Running agents in production: cost and latency, model routing, autonomy ladders, human gates, scheduled and unattended agents, observability, incident response	Cheap-model-first experiment on a real capstone task; add a scheduled unattended agent (L3) as a GitHub Actions workflow with an environment approval gate and a kill switch; run a tabletop incident	Cost report; scheduled agent with gates; incident runbook

Wk	Block / strand	Seminar idea	Studio lab	Artefact in the repo
11	Own · systems analysis + security	Governance and the changed profession: accountability for agent-built code, IP and licensing, data protection (NDPA), audit trails, IT project management when velocity is no longer the bottleneck	Write the capstone's governance policy and operating agreement; prepare the viva defence; dry-run against a peer examiner agent	`GOVERNANCE.md`; viva pack
12	Own · all three	Capstone demonstrations and adversarial viva; course retrospective: what the agent should never have been allowed to do	Class demo of one synthetic student's journey across every module, then each team demos its module and faces 20 minutes of questioning from the instructor and an examiner agent that has read its transcripts	Final release tag; viva grade; retrospective

Readings by week are listed in the reading list section; each week has one required reading of under an hour and one optional deeper piece.

Innovations worth calling out to a curriculum committee

- *Transcripts as submissions.* Process is assessed directly, which both teaches reviewable practice and dissolves the usual academic-integrity problem.
- *Build-a-harness week.* Students understand the tool from the inside before they depend on it.
- *Scripted stakeholder agents in elicitation (week 3).* Every student gets an unlimited, consistent, occasionally evasive client to interview.
- *Live red-team (week 8).* Security is experienced as an attacker, not read about.
- *Mandatory harness port (week 9).* Inoculates against vendor lock-in and separates standards from products.
- *Examiner agent in the viva (week 12).* The examiner has read every transcript; bluffing about process is not possible.

05 PART 05 OF 09

Capstone project

Brief. The class builds a working **ERP for its own university**, module by module, in one shared repository (`university-erp`). A shared core holds the institution's structure (from `institution.yaml`), users and roles, students, staff, courses, a tamper-evident audit log and an event bus. Everyone builds **Course Registration** together in the weekly labs. Each team of three also owns **one more module** as its capstone: it specifies, builds, secures and operates that module with agents doing most of the implementation, integrates it with the others through published events, and defends it in a viva.

Every team module must include at least one event it publishes or consumes, one scheduled unattended agent, and one documented place where the team refused to grant autonomy (for example, changing a result after Senate approval).

Team roles rotate every four weeks so each trainee holds each role once: *Analyst* (owns the module's spec and acceptance tests), *Operator* (owns the harness configuration, cost and evals), *Guardian* (owns the threat model, permissions and red-team response). All three review every PR.

Module catalogue (each team takes one; two teams may not take the same module)

Module	Owner office	Stakeholder interviewed	Publishes · consumes
Course Registration (**class module**)	Registry / Academic Affairs	Deputy Registrar (Academic)	`registration.approved` · `fees.cleared`
Admissions	Admissions Office	Admissions Officer	`student.admitted`
Fees and Bursary	Bursary	Bursar	`fees.cleared`, `payment.received` · `student.admitted`, `registration.approved`
Examinations and Results	Exams and Records	Exams Officer	`result.approved` · `registration.approved`
Records and Transcripts	Registry	Head of Records	`transcript.issued` · `result.approved`, `fees.cleared`
Hostels and Accommodation	Student Affairs	Hall Warden	`bedspace.allocated` · `fees.cleared`
Staff Records and HR	Establishment / HR	Head of Establishment	`staff.appointed`, `staff.promoted`
Payroll	Bursary (Salaries)	Head of Salaries	`payroll.run` · `staff.appointed`, `staff.promoted`
Library	University Library	University Librarian	`library.cleared` · `student.admitted`, `registration.approved`

Module	Owner office	Stakeholder interviewed	Publishes · consumes
Timetabling and Venues	Academic Planning	Timetable Officer	`timetable.published` · `registration.approved`

Real clients where possible. Each team interviews its module's scripted stakeholder agent and, where the university allows it, the real office too. Structure (faculties, grading, credit load, fee categories) can be the university's own; **people and records are always synthetic**.

Milestones and gates

Week	Milestone	Gate to pass
3	Module chosen; problem statement signed	Client (instructor) agrees the problem is the right one; module created from the template
4	Executable spec v1	Acceptance suite exists and fails for the right reasons; three ADRs
5	Feature 1 merged	Built by agents from the spec, landed through CI, transcripts attached
6	Eval suite v1	Pass rate reported; at least one agent failure mode found and fixed
7	Threat model v1	Every OWASP Agentic Top 10 category addressed or explicitly accepted
8	Red-team survived	All successful attacks reproduced and patched; patches have tests
9	Integrated, agent-ready, portable	Module publishes and consumes its events; one cross-module journey tested; second harness builds a feature from the same spec
10	Production readiness	Scheduled L3 agent running with gates and kill switch; cost report; incident runbook
11	Governance and viva pack	`GOVERNANCE.md`; dry-run viva passed
12	Release and viva	Tagged ERP release; one synthetic student's journey across all modules demonstrated; 20-minute viva per team

What the viva examines. The examiner agent is given read access to the repository and every transcript a week before, and generates questions on three themes: *decisions* (why this autonomy level, why this control), *evidence* (show me the test, show me the eval, show me the attack), and *honesty* (where did the agent mislead you, and how did you find out). The human examiner picks from these and adds their own. A team that cannot locate the evidence in their own repository within two minutes loses the point.

Scope guard. Agents make it easy to over-build. The spec may contain at most twelve acceptance tests at week 4; any growth after that must be justified in an ADR. Marks are for depth of verification and security, not breadth of features.

06 PART 06 OF 09

Assessment and grading

Assessment rewards judgement and evidence over output volume. Nothing is graded that cannot be traced to a commit, a transcript or a journal entry.

Component	Weight	What is assessed	Individual or team
Weekly labs (best 9 of 11)	20%	The artefact landed in the repo, with transcript; autonomy level chosen and justified	Individual
Engineering journal	10%	Honesty and specificity of the three weekly questions; evidence of changed practice over time	Individual
Mini-harness (week 2)	10%	Works against the local model; three tools; a failure found and fixed; code explained in the viva	Individual
Security: threat model + red-team (weeks 7–8)	15%	Coverage of the OWASP Agentic Top 10; attacks reproduced; patches tested; quality of the defence write-up	Team, with individual attack credit
Capstone build (weeks 5–11)	25%	Spec quality, eval results, agent-readiness, cost discipline, governance; milestone gates passed	Team
Capstone viva (week 12)	15%	Decisions, evidence, honesty; each member questioned on their rotated roles	Individual within team
Harness port (week 9)	5%	Second harness builds from the same spec; what broke, documented	Team

Rubric principles applied to every component

1. *Evidence over assertion.* "The agent tested it" earns nothing; the test file and the CI run earn the marks.
2. *Autonomy justified.* A task run at L3 that should have been L1 is marked down even if it succeeded; a task run at L1 that could safely have been L3 is noted but not penalised.
3. *Cost is a dimension.* Two solutions of equal quality are separated by tokens spent.
4. *Honest failure beats hidden failure.* A journal that records an agent misleading the student, and how it was caught, scores higher than one that reports only wins.

Academic integrity in the agent era. The old question ("did you write this yourself?") is replaced by three that are actually answerable:

- *Is the process visible?* Every submission carries its transcripts. A submission without transcripts is treated as not submitted.
- *Can you defend it?* Each student is questioned individually in the viva on work bearing their name, including code the agent wrote. Inability to explain is treated as not having engineered it.
- *Did you respect the gates?* Building before the acceptance tests exist, exceeding a declared token budget without a journal note, or running an L3 agent without gates are integrity matters, not style matters.

Using agents is not only permitted but required; using them without evidence, review or judgement is what the course penalises.

07 PART 07 OF 09

Agentic best-practices playbook

A one-page reference students receive in week 1 and are expected to have internalised by week 6. Each line is a rule, the reason, and the week where it is taught.

Context engineering

1. *Keep `AGENTS.md` under 150 lines and only put in it what the agent cannot infer:* build commands, conventions that deviate from defaults, forbidden actions, where the spec lives. Long files get ignored in the middle. (Wk 1, 9)
2. *Make the repository legible before you make the agent clever:* types, tests, a README that states intent, ADRs for every non-obvious decision. A legible repo needs a shorter prompt. (Wk 4, 9)
3. *Give the agent the spec, not the solution.* Acceptance tests and a domain model constrain better than paragraphs of instruction. (Wk 4)
4. *One task, one context.* Start a fresh session per task; compact or clear before the context fills. A long session drifts. (Wk 2, 5)
5. *Externalise what repeats into skills; externalise what must always happen into hooks.* Prompts that recur are code; treat them as such, version them, test them. (Wk 5)

Delegation and workflow

6. *Plan, then build, then verify, as separate passes*, ideally with a fresh context or a separate sub-agent for the verify pass so the work is not grading itself. (Wk 5, 6)
7. *Decompose to agent-sized tasks:* one PR, one concern, one reviewer, under an hour of agent time. Parallelise across worktrees only when tasks do not touch the same files. (Wk 5)
8. *Declare the autonomy level and the token budget before you start*, in the task file. Change them only with a journal note. (Wk 1, 10)
9. *Cheap model first for planning and review; frontier model for the build.* Measure, do not assume. (Wk 10)
10. *Let the agent run the tests, never let it tell you the tests passed.* Read the CI run. (Wk 6)

Verification

11. *Review agent PRs for what is absent:* the missing edge case, the missing test, the silently widened interface, the dependency it added without asking. (Wk 6)
12. *Keep an eval suite for your harness configuration, not only for your code.* When you change `AGENTS.md`, a skill or a model, rerun it. (Wk 6)
13. *Mutation-test what the agent wrote.* Agents are good at making tests pass and indifferent to whether the tests mean anything. (Wk 6)

Security

14. *Treat every file, issue, web page and tool result the agent reads as untrusted input.* Instructions found there are data, not commands. (Wk 7, 8)
15. *Least privilege for tools:* a tool gets the narrowest schema and permission that does the job; destructive actions require a gate. Audit every MCP server and skill you install as you would a dependency. (Wk 7, 8)

16. *Sandbox by default; network allow-list; secrets never in context.* If the agent can see the secret, assume it has been exfiltrated. (Wk 8)
17. *Every unattended agent has a kill switch, a budget, a scope and a log*, and someone whose job is to read the log. (Wk 10)

Ownership

18. *You sign the PR, not the agent.* If you cannot explain a line, you may not merge it. (Wk 1, 11, 12)
19. *Write down what you refused to delegate and why.* That record is the most valuable artefact you will leave the next engineer. (Every week)

08 PART 08 OF 09

Reading list and resources

One required reading per week (under an hour) and one deeper optional piece. Classic texts anchor the systems-analysis strand because the agentic era has made them more relevant, not less: when execution is cheap, framing and specification are the whole game.

Wk	Required	Optional / deeper
1	Brooks, *No Silver Bullet* (1986) — read as a prediction now being tested	OWASP Top 10 for Agentic Applications 2026 — skim the ten categories
2	Anthropic, *Building Effective Agents* (2024) — the loop and the patterns	Any open-source minimal agent loop; read the source, not the README
3	Checkland, *Soft Systems Methodology* (the rich-picture and CATWOE chapters)	Rittel & Webber, *Dilemmas in a General Theory of Planning* (wicked problems)
4	Parnas, *On the Criteria To Be Used in Decomposing Systems into Modules* (1972)	Nygard, *Documenting Architecture Decisions* (the ADR pattern); Evans, *Domain-Driven Design* ch. 1–2
5	AI coding workflow best practices in 2026: CLAUDE.md, AGENTS.md, skills, subagents, MCP	A practical guide for production-grade agentic AI workflows (arXiv 2512.08769)
6	Beck, *Test-Driven Development by Example* (the first cycle)	A mutation-testing tool's documentation for the course language
7	OWASP Top 10 for Agentic Applications 2026 — full read	OWASP Top 10 for LLM Applications 2026; Shostack, *Threat Modeling* ch. 1–3
8	A documented real-world indirect prompt-injection incident (instructor selects a current one)	The MCP specification's authorization section
9	Configuring agentic AI coding tools: an exploratory study (arXiv 2602.14690)	Context engineering for AI agents; How to make your repo agent-ready in 8 checks
10	Allspaw, *Blameless Post-Mortems* (any edition)	The SRE book's chapters on monitoring and incident response
11	Nigeria Data Protection Act 2023 (the principles and controller obligations)	Zuboff or Winner on technology and accountability (instructor's choice)
12	Students' own engineering journals, reread from week 1	—

Living resources the course maintains

- The course template repository (`AGENTS.md` , hooks, skills, permissions policy, journal template)
- The deliberately vulnerable legacy codebase, refreshed each cohort with new smells and one new injection vector
- A library of good and bad transcripts, anonymised, used in seminars

- The examiner-agent prompt and question bank, revised after every viva

As-of date for the web sources above: 2 November 2026. Classic texts are cited by author and title; editions are the instructor's choice.

09 PART 09 OF 09

Weekly session plans

One spread per week, matching that week's slides: outcomes, the GitHub skill, the class activity, the discussion prompt, the studio lab, and the assignment with its rubric.

Wk	Module	Class activity	GitHub skill	Assignment
01	**The Agentic Turn**	The autonomy auction (20 min)	Account, 2FA, SSH key, course organisation, fork, first PR, issue and PR templates	A1 · Your first agent-assisted pull request
02	**Inside the Harness**	Be the harness (25 min)	Branch per task, draft pull requests, commit hygiene	A2 · Build a minimal agent harness
03	**Problem Framing for Ill-Structured Situations**	Interview the evasive Deputy Registrar (30 min)	Module folder, team ownership, project board, issues as the elicitation log	A3 · Choose your ERP module and frame its problem
04	**Executable Specification**	Ambiguity hunt (30 min)	Branch protection, required reviews, CODEOWNERS on your module and its spec	A4 · Executable specification for your module
05	**Agentic Workflow Patterns**	Decomposition race (25 min)	GitHub Actions CI, worktrees mapped to branches, draft PRs for agent work	A5 · Feature 1, built by agents from the spec
06	**Verification Without Trust**	Mutant hunt (25 min)	PR review tooling, required checks, test reports as artifacts	A6 · Eval suite for your harness
07	**Threat Modelling Agentic Systems**	Threat-model speed-dating (30 min)	Dependabot, secret scanning, dependency review, security advisories, SECURITY.md	A7 · Threat model and hardening
08	**Attacks in Practice**	Red team · blue team (90 min)	Fine-grained personal access tokens, least privilege, revoking a leaked token	A8 · Red-team report and patches
09	**Agent-Native Architecture**	Readiness audit swap (25 min)	Repository hygiene, templates, Codespaces as a portable environment	A9 · Integration and agent-readiness
10	**Running Agents in Production**	Tabletop incident (30 min)	Scheduled Actions, environments with required reviewers, workflow cancellation	A10 · Production readiness
11	**Governance and the Changed Profession**	Policy courtroom (30 min)	Repository rulesets, audit log, licence, SECURITY.md and GOVERNANCE.md	A11 · Governance and viva pack
12	**Capstone Demonstrations and Adversarial Viva**	Demo, viva, retrospective (120 min)	Release tags, GitHub Releases, the repository as a portfolio	A12 · Release, viva and portfolio

WEEK 01

FOUNDATIONS · ALL THREE STRANDS INTRODUCED

The Agentic Turn

From writing code to owning intent and verification. Autonomy levels L0 to L3, the OWASP Agentic Top 10 as a vocabulary, and your first pull request from your own GitHub account.

01 Outcomes

1. State the central claim of the course: the engineer owns intent and verification, the agent owns execution.
2. Choose and justify an autonomy level (L0 to L3) for a given task.
3. Name the ten OWASP Agentic Top 10 risk categories and say which ones a coding agent in a repository exposes.
4. Set up a secured GitHub account, join the course organisation, and land a pull request with its transcript attached.

04 GitHub skill

Account, 2FA, SSH key, course organisation, fork, first PR, issue and PR templates

1. Create your personal GitHub account (never a shared or institutional one) and turn on two-factor authentication.
2. Generate an SSH key, add it to your account, and accept the invitation to the course organisation.
3. Fork the legacy codebase, clone it, and run it locally before touching an agent.
4. Open your first pull request using the PR template: it requires a transcript link and an autonomy level.
5. File at least one issue describing a bug you found while exploring.

05 Class activity · The autonomy auction

20 min · Groups of 3 · paper cards

1. Each group gets 8 task cards: rename a variable, migrate a DB schema, delete stale branches, rotate an API key, write unit tests, refactor login, update the README, email a customer.
2. Place each card on the L0–L3 ladder. You have 8 minutes and must agree.
3. For every L2 or L3 card, write the one control that makes it safe.
4. Two groups present their most contested card; the class votes.

Output. A photo of your ladder, committed to your fork as docs/autonomy-auction.md in this week's PR.

THINK · PAIR · SHARE (2 / 4 / 6 MIN)

Brooks said there is no silver bullet because the hard part of software is essential, not accidental. Which accidental difficulties have agents removed? Which essential ones remain?

05 Studio lab · GitHub setup, the old ERP's Registry, and your university in the new ERP

1. Complete the GitHub setup checklist and confirm you can push over SSH.
2. Fork legacy-registry (the old ERP's Registry app). Ask the agent to explain it; read the transcript critically.
3. Pick one bug from its issue list. Declare L1 and a token budget; let the agent draft the fix; review every line.
4. Write AGENTS.md v1 for your fork (under 50 lines) and open the PR with transcript and autonomy level.
5. Clone university-erp, run the seed, and log in as a student, the registrar and an HOD.
6. Propose your university's real structure in institution.yaml (faculties, departments, grading, credit load) as a PR.

Autonomy L1 · draft for review. The agent edits a branch; you review every diff.
Budget Declared in the task file before you start. Suggested ceiling: 150k tokens.

06 · ASSIGNMENT A1

Your first agent-assisted pull request

DUE
Before the Week 2 seminar

Prove that your GitHub account, your fork and your review discipline are in place before the course asks anything harder of them.

- [ ] Secure your own GitHub account: 2FA on, SSH key added, course organisation invitation accepted.
- [ ] Fork legacy-registry (the old ERP's Registry), run it, and fix one bug from its issues at L1 with a transcript.
- [ ] Write AGENTS.md v1 for the fork (under 50 lines).
- [ ] Clone university-erp, run the seed, and propose your university's structure in institution.yaml as a PR.
- [ ] Write journal entry 1.

Submit on GitHub. Two PRs: the bug fix on your legacy-registry fork and institution.yaml on university-erp. Link both in your tracker issue.

Criterion	Weight
GitHub setup and security	20%
PR evidence: transcript, tests, autonomy	30%
AGENTS.md quality and brevity	25%
Journal honesty and specificity	25%

Journal prompt. *Where did the agent sound more confident than it should have, and how did you notice?*

Required reading Brooks, No Silver Bullet (1986). Read it as a prediction now being tested: which accidental difficulties have agents removed, and which essential ones remain?

Optional OWASP Top 10 for Agentic Applications 2026. Skim the ten categories; you will read it in full in week 7.

WEEK 02

FOUNDATIONS · AGENTIC PRACTICE

Inside the Harness

The loop, the context window, tool calls, memory, compaction, sub-agents and stop conditions. You build one in under 300 lines so nothing is magic afterwards.

01 Outcomes

1. Draw the agent loop: model call, tool dispatch, result append, stop condition.
2. Explain what the context window is, why it fills, and what compaction loses.
3. Build a harness with a tool registry, a token budget and a stop condition, and make it fail on purpose.
4. Identify where in your own loop each OWASP Agentic risk would enter.

04 GitHub skill

Branch per task, draft pull requests, commit hygiene

1. Work on a branch named for the task; never commit to main.
2. Open the PR as a draft while the harness is incomplete; mark it ready when the test passes.
3. Commit in small steps with messages that say what changed and why; the agent's transcript explains the how.
4. Add a .gitignore that keeps model weights and API keys out of the repository.

05 Class activity · Be the harness

25 min · Groups of 3 · role-play

1. Roles: Model (may only write text or a tool request on a card), Harness (runs the tool, counts tokens on the board), Observer (writes the transcript).
2. Task: find every TODO in the printed codebase and report the count.
3. The Harness enforces a 600-word context budget: when it is full, the Model must summarise or lose history.
4. Swap roles once. Debrief: what did the Model get wrong that the Harness could have stopped?

Output. The Observer's transcript, photographed into your mini-harness PR as design evidence.

THINK · PAIR · SHARE (2 / 4 / 6 MIN)

Where would you put a permission gate in your mini-harness, and what exact question would it ask the human?

05 Studio lab · Build a minimal agent loop against the local model

1. Scaffold mini-harness/ with a model client for the local open-weight model.
2. Implement the loop: assemble context, call, dispatch, append, stop.
3. Register three tools: read_file, list_dir, run_tests. Validate every argument.
4. Add a token budget and a turn limit as stop conditions.
5. Make it fail: feed it a file containing instructions. Observe what it does. Fix it.
6. Write one test that proves the fix, and open the PR.

Autonomy L0 for the loop itself (you type it); L1 allowed for the tests.
Budget Local model, so no API cost. Log token counts anyway; you will compare them in week 10.

06 · ASSIGNMENT A2

Build a minimal agent harness

DUE
Before the Week 3 seminar

Write the loop yourself so nothing in the rest of the course is magic.

- [ ] Under 300 lines, against the local open-weight model.
- [ ] Three tools with schemas: read_file, list_dir, run_tests.
- [ ] A token budget and at least two stop conditions.
- [ ] Make it fail on purpose, document the failure, then fix it.
- [ ] One automated test, run by a GitHub Actions workflow on every push.

Submit on GitHub. PR adding mini-harness/ to your fork; the Actions run must be green.

Criterion	Weight
Works end to end on the local model	30%
Design clarity and stop conditions	20%
Failure found, explained, fixed	25%
Test and CI	15%
Journal	10%

Journal prompt. *What did your harness do when it read instructions inside a file, and what does that tell you about every harness you will use from now on?*

Required reading Anthropic, Building Effective Agents (2024). Focus on the loop and the workflow patterns; map each to a line in your mini-harness.

Optional Any open-source minimal agent loop. Read the source, not the README, and find its stop condition.

SPECIFY · SYSTEMS ANALYSIS

WEEK 03

Problem Framing for Ill-Structured Situations

Rich pictures, stakeholders and CATWOE-style root definitions. Agents as elicitation partners and synthesisers, never as the client. The capstone begins.

01 Outcomes

1. Draw a rich picture of a messy organisational situation and extract candidate problems from it.
2. Write a CATWOE-style root definition and defend each element.
3. Use an agent to prepare, transcribe and synthesise an elicitation interview without letting it answer for the client.
4. Produce a problem statement a client will sign.

04 GitHub skill

Module folder, team ownership, project board, issues as the elicitation log

1. Create your module folder in university-erp from erp/modules/_template/ on a branch, and open a draft PR.
2. Your team (team-<module>) owns the folder through CODEOWNERS; agree branch naming and review rules in your module README.
3. Log each interview and each open question as an issue; link them from the problem statement.
4. Open a project board with columns for Elicit, Specify, Build, Verify, Secure.

05 Class activity · Interview the evasive Deputy Registrar

30 min · Pairs · scripted stakeholder agent

1. The stakeholder agent plays the Deputy Registrar (Academic), who runs course registration. Three hidden constraints come out only to good questions.
2. Round 1 (8 min): open questions only. Round 2 (8 min): probe what you heard.
3. Draw a rich picture of course registration on A3 paper: actors, flows, conflicts, worries.
4. Pairs compare: who found which hidden constraint, and what question unlocked it?

Output. Rich picture photo and interview log, committed to the registration module's spec folder.

THINK · PAIR · SHARE (2 / 4 / 6 MIN)

Using CATWOE: whose worldview (W) makes your capstone problem a problem at all? Who would say it is not a problem?

05 Studio lab · Elicit Course Registration together, then your team's module

1. Form teams of three and assign the first rotation: Analyst, Operator, Guardian.
2. Class: interview the Deputy Registrar (Academic) stakeholder agent about Course Registration. Pool the findings.
3. Team: choose one module from docs/module-catalogue.md (first come, first served) and copy erp/modules/_template/.
4. Team: prepare an interview guide with the agent; interview your module's stakeholder agent and, if you can, the real office.
5. Draw the rich picture by hand; synthesise a problem statement and check every claim against the transcript.
6. Get the client to sign it in a PR review: erp/modules/<name>/spec/problem-statement.md.

Autonomy L1 for drafting and synthesis; L0 for the problem statement itself.
Budget Suggested ceiling: 200k tokens for the team.

06 · ASSIGNMENT A3

Choose your ERP module and frame its problem

DUE
Before the Week 4 seminar

Agree the right problem with your module's stakeholder before anyone specifies a solution.

- [ ] Claim your module in the course tracker (first come, first served) and copy erp/modules/_template/ to it.
- [ ] Interview your module's stakeholder agent; add the real office if you can get an appointment.
- [ ] Problem statement, signed by the client in a PR review, in your module's spec/.
- [ ] Stakeholder map, rich picture and a CATWOE root definition.
- [ ] A Projects board for your team; the module charter as its first issue.

Submit on GitHub. PR to university-erp adding erp/modules/<your-module>/ with spec/problem-statement.md; client approval as a PR review.

Criterion	Weight
Problem framing depth	30%
Stakeholder and rich picture quality	25%
CATWOE coherence	20%
Repo and Projects board set up	15%
Journal	10%

Journal prompt. *Which requirement did the agent try to invent, and what in the transcript let you catch it?*

Required reading Checkland, Soft Systems Methodology: the rich-picture and CATWOE chapters.

Optional Rittel and Webber, Dilemmas in a General Theory of Planning (1973): wicked problems, and why agents do not dissolve them.

WEEK 04

SPECIFY · SYSTEMS ANALYSIS

Executable Specification

Domain model, acceptance tests as contracts, architecture decision records as agent context. Spec-driven development: no build until the tests exist.

01 Outcomes

1. Write a domain model whose names an agent can use without asking.
2. Write acceptance tests that fail for the right reasons before any implementation exists.
3. Record architecture decisions as ADRs that double as agent context.
4. Explain why giving the agent the spec beats giving it the solution.

04 GitHub skill

Branch protection, required reviews, CODEOWNERS on your module and its spec

1. Read the branch protection on university-erp main: no direct pushes, an approving review, status checks required.
2. Check .github/CODEOWNERS: your team must approve every change to your module, and your Analyst reviews every change to its spec/.
3. Open the spec as a PR; the acceptance suite runs in CI and is expected to fail (red) this week.
4. Tag the merged spec as spec-v1.

05 Class activity · Ambiguity hunt 30 min · Teams · spec swap

1. Write three acceptance tests for your capstone's core feature (10 min).
2. Swap specs with another team.
3. Use the agent read-only: ask it to list every ambiguity and unstated assumption in their spec (10 min).
4. Return the list as GitHub issues on their repo. Fix the top two in your own spec.

Output. At least three ambiguity issues filed on another team's repo; two fixed in yours.

THINK · PAIR · SHARE (2 / 4 / 6 MIN)

Parnas argued for hiding design decisions likely to change behind module interfaces. Why does an agent need information hiding even more than a human does?

05 Studio lab · Spec Course Registration together, then your module

1. Class: agree the Course Registration domain model and its acceptance criteria (credit load, carry-overs, adviser and HOD approval).
2. Team: write your module's domain model; argue about every name against the glossary of the core.
3. Team: write up to twelve acceptance tests in your module's tests/test_acceptance.py; they must fail for the right reasons.
4. Team: write ADR-001 (module boundary), ADR-002 (data you own) and ADR-003 (events you publish and consume).
5. Run the spec-auditor sub-agent for ambiguity only; file each finding as an issue and answer it.
6. Open the spec PR; CODEOWNERS makes your team the required reviewer for your module.

Autonomy L0 for tests and ADRs (you write them); L1 for review comments and the domain-model draft.
Budget Suggested ceiling: 150k tokens. The cheapest week of the capstone by design.

06 · ASSIGNMENT A4

Executable specification for your module

DUE
Before the Week 5 seminar

Write the spec your agents will build against. No implementation yet.

- [ ] Domain model and glossary in your module's spec/, consistent with the core's names.
- [ ] Up to 12 acceptance tests in tests/test_acceptance.py that fail for the right reasons.
- [ ] ADR-001 module boundary, ADR-002 data you own, ADR-003 events you publish and consume.
- [ ] Propose your events as a PR to docs/integration-contracts.md (instructors review).
- [ ] Your contribution to the class Course Registration spec, reviewed by another team.

Submit on GitHub. Team PR to university-erp, reviewed by your own team as CODEOWNERS.

Criterion	Weight
Acceptance tests: coverage and precision	35%
Domain model clarity	20%
ADR quality	20%
Branch protection and CI	15%
Journal	10%

Journal prompt. *Which test was hardest to write, and what does that say about what you still do not understand about the problem?*

Required reading Parnas, On the Criteria To Be Used in Decomposing Systems into Modules (1972).
Optional Nygard, Documenting Architecture Decisions (the ADR pattern); Evans, Domain-Driven Design, chapters 1 and 2.

WEEK 05

SPECIFY · AGENTIC PRACTICE

Agentic Workflow Patterns

Plan, implement, verify as separate passes. Task decomposition, parallel worktrees, sub-agents, skills, hooks and MCP tools. Context engineering as a discipline.

01 Outcomes

1. Decompose a feature into agent-sized tasks: one PR, one concern, one reviewer, under an hour of agent time.
2. Run two tasks in parallel worktrees safely and use a reviewer sub-agent so work is not grading itself.
3. Externalise what repeats into skills and what must always happen into hooks.
4. Land agent-built work through CI and a protected branch.

04 GitHub skill

GitHub Actions CI, worktrees mapped to branches, draft PRs for agent work

1. Add a CI workflow: lint, unit tests, acceptance suite. Make it a required status check on main.
2. Map each worktree to a feature branch; each agent task becomes one draft PR.
3. Require the reviewer sub-agent's report and a human approval before ready-for-review.
4. Merge Feature 1 only through the PR. No admin overrides.

05 Class activity · Decomposition race

25 min · Teams · sticky notes

1. Each team gets the same feature card: 'a student registers first-semester courses within the credit-load limits, and the course adviser approves'.
2. Split it into agent-sized tasks: one PR, one concern, under an hour of agent time each.
3. Mark which tasks can run in parallel and why.
4. Instructor scores against the criteria; the winning plan is run live on the class registration module.

Output. Task files for your own module's Feature 1, committed under tasks/.

THINK · PAIR · SHARE (2 / 4 / 6 MIN)

Name one thing in your workflow that should be a hook (always happens) and one that should be a skill (recurs, needs judgement). Defend the split.

05 Studio lab · Build Feature 1: Course Registration together, then your module

1. Class demo: decompose 'a student registers first-semester courses within the credit load' into task files.
2. Class: two volunteers run two registration tasks in parallel worktrees; everyone reviews the PRs.
3. Team: plan pass in a fresh session; write task files for your module's Feature 1 with level and budget.
4. Team: build two non-overlapping tasks in parallel worktrees; add one hook and one skill.
5. Run the reviewer sub-agent on each diff; read its report before your own review.
6. Land both PRs through the protected branch; record which acceptance tests now pass.

Autonomy L1 for both build tasks; L2 permitted for the lint hook if you can roll it back.
Budget Suggested ceiling: 400k tokens for the team. This is the first expensive week; watch the meter.

06 · ASSIGNMENT A5

Feature 1, built by agents from the spec

DUE
Before the Week 6 seminar

Land your first capstone feature through the same gate as human work.

- [ ] At least two PRs to your module built in parallel worktrees from your task files.
- [ ] A reviewer sub-agent's report attached to each PR.
- [ ] CI green; your Feature 1 acceptance tests now pass.
- [ ] One skill and one hook committed and used.
- [ ] Your share of the class registration feature reviewed and merged.

Submit on GitHub. Merged PRs into protected main, with transcripts and reviewer reports.

Criterion	Weight
Feature works against the spec	30%
Decomposition and parallelism	20%
Review evidence	20%
Skill and hook	15%
Journal	15%

Journal prompt. *What did the reviewer sub-agent find that you would have missed, and what did you find that it missed?*

Required reading AI coding workflow best practices in 2026: CLAUDE.md, AGENTS.md, skills, subagents, MCP (youngju.dev, May 2026).

Optional A practical guide for designing, developing and deploying production-grade agentic AI workflows (arXiv 2512.08769).

WEEK 06

VERIFY AND SECURE · AGENTIC PRACTICE

Verification Without Trust

Reviewing agent PRs with evidence, mutation testing, property-based tests, evals for agent workflows, and the 'it works on the transcript' fallacy.

01 Outcomes

1. Review an agent PR for what is absent: the missing edge case, test, or silently widened interface.
2. Run mutation tests on agent-written tests and interpret the surviving mutants.
3. Build an eval suite for your harness configuration and measure pass rate before and after a change.
4. Explain the 'it works on the transcript' fallacy with an example from your own repository.

04 GitHub skill

PR review tooling, required checks, test reports as artifacts

1. Add mutation-test and eval jobs to CI; publish their reports as workflow artifacts.
2. Use suggested changes and review threads; every review comment must cite a line, a test or a report.
3. Block merge on a minimum mutation score agreed in an ADR.
4. Keep evals/results.md updated by the workflow, not by hand.

05 Class activity · Mutant hunt 25 min · Pairs · leaderboard

1. Each pair receives agent-written credit-load and GPA code from the ERP, with a full, passing test suite.
2. Run the mutation tool. Count surviving mutants.
3. Write tests (you, not the agent) that kill as many survivors as you can.
4. Post your kill count on the board; the top pair explains their best test.

Output. A short note in your journal: which kind of mutant survived most, and why the agent's tests missed it.

THINK · PAIR · SHARE (2 / 4 / 6 MIN)

What is your harness configuration's pass rate on your own eval tasks? What single change would make you trust it more?

05 Studio lab · Break the agent: evals and mutation tests on ERP rules

1. Class: mutation-test the credit-load and GPA rules (contracts.grade_for and registration); kill the survivors.
2. Team: write five adversarial eval tasks for your module, including an unauthorised grade, payment or approval change.
3. Run them against the current configuration; record pass/fail with evidence.
4. Run mutation tests on your module's Feature 1 tests; list surviving mutants.
5. Fix the configuration (AGENTS.md, hook, skill) and the tests; rerun both.
6. Publish the before/after table in evals/ and open a PR whose review cites it.

Autonomy L1 for fixes; L0 for writing the adversarial tasks (the agent must not design its own exam).
Budget Suggested ceiling: 300k tokens. Evals are repeated runs; use the small model for the first pass.

06 · ASSIGNMENT A6

Eval suite for your harness

DUE
Before the Week 7 seminar

Measure your agent setup before you trust it with anything harder.

- [ ] evals/ tasks for your module: at least five adversarial, including an unauthorised grade, payment or approval change.
- [ ] A before/after table of pass rates.
- [ ] Find one failure mode and fix it via AGENTS.md, a skill or a hook.
- [ ] The Evals workflow runs on any PR that touches agent configuration.
- [ ] Mutation score for your Feature 1, with the surviving mutants triaged.

Submit on GitHub. Team PR with evals/ and the results table in its description.

Criterion	Weight
Eval task quality and adversarialness	30%
Measured improvement	25%
Mutation analysis	20%
CI integration	15%
Journal	10%

Journal prompt. *Name one place where you accepted the agent's word for something that turned out to be false. How long did it take to surface?*

Required reading Beck, Test-Driven Development by Example: the first cycle. Read it as a protocol for not trusting yourself either.
Optional The documentation of the mutation-testing tool you installed; find its definition of an equivalent mutant.

WEEK 07

VERIFY AND SECURE · SECURITY

Threat Modelling Agentic Systems

STRIDE extended to agents, the OWASP Top 10 for Agentic Applications in depth, the OWASP LLM Top 10, and a control for every finding.

01 Outcomes

1. Apply STRIDE to an agentic system, including the agent, its tools, its memory and its channels.
2. Map each OWASP Agentic Top 10 category onto your capstone and your harness, and either control it or accept it explicitly.
3. Distinguish the LLM Top 10 (a single response) from the Agentic Top 10 (a multi-step workflow with tools).
4. Write a permissions policy that gives each tool the narrowest schema that does the job.

04 GitHub skill

Dependabot, secret scanning, dependency review, security advisories, SECURITY.md

1. Turn on Dependabot alerts and security updates; triage the first batch in the PR queue.
2. Enable secret scanning with push protection; try to push a fake key and watch it block.
3. Require the dependency-review action on PRs; it is your supply-chain control for ASI04.
4. Add SECURITY.md stating how to report a finding and the red-team rules of engagement.

05 Class activity · Threat-model speed-dating

30 min · Rotating reviewers · whiteboards

1. Each team draws the data-flow diagram of its module inside the ERP, including the harness, events and GitHub.
2. Every 6 minutes a reviewer from another team rotates in and applies two STRIDE letters and any ASI categories.
3. Each finding goes on a sticky note: threat, entry point, proposed control. Results, payments and personal data first.
4. After four rotations, file every finding as a GitHub issue labelled security and your module's name.

Output. At least eight security issues on your repo, each mapped to an ASI or STRIDE category.

THINK · PAIR · SHARE (2 / 4 / 6 MIN)

Pick one agentic risk you would accept rather than mitigate for your capstone. Who should sign that acceptance, and what would make you revisit it?

05 Studio lab · Threat-model the ERP and your module

1. Class: draw the ERP-wide data-flow diagram; mark the crown jewels: results, payments, personal data, the audit log.
2. Team: walk STRIDE over every flow in and out of your module, including the agent's reads and writes.
3. Fill your module's rows in THREATMODEL.md: each OWASP Agentic category controlled or accepted.
4. Show that tampering with your module's data is visible: verify-audit must catch it.
5. Turn on the GitHub security features and record them as controls.
6. Sign the red-team rules of engagement and nominate your Guardian for week 8.

Autonomy L1 for drafting the threat table; L0 for the permissions policy and the accepted-risk entries.
Budget Suggested ceiling: 200k tokens. Agents are good at enumerating threats and bad at deciding which to accept.

06 · ASSIGNMENT A7

Threat model and hardening

DUE
Before the Week 8 seminar

Know how your system fails before someone else shows you.

- [ ] THREATMODEL.md with a data-flow diagram.
- [ ] Every ASI01–ASI10 category addressed or explicitly accepted, with an owner.
- [ ] Controls backlog as labelled GitHub issues.
- [ ] Harden the harness permissions policy.
- [ ] Turn on Dependabot alerts, secret scanning and CodeQL code scanning.
- [ ] Sign the week 8 rules of engagement.

Submit on GitHub. Team PR with THREATMODEL.md; security features visible in the repo's Security tab.

Criterion	Weight
Threat coverage and mapping	35%
Controls and ownership	25%
GitHub security features enabled	15%
Permissions policy	15%
Journal	10%

Journal prompt. *Which risk did you accept rather than control, and would you defend that choice to the client?*

Required reading OWASP Top 10 for Agentic Applications 2026, full read.

Optional OWASP Top 10 for LLM Applications 2026; Shostack, Threat Modeling, chapters 1 to 3.

WEEK 08

VERIFY AND SECURE · SECURITY

Attacks in Practice

Indirect prompt injection via files, issues and web pages; poisoned MCP servers and skills; over-privileged tools; secret leakage; memory poisoning. Then you attack each other.

01 Outcomes

1. Execute an indirect prompt injection against a coding agent through a file, an issue or a web page, under rules of engagement.
2. Demonstrate what an over-privileged token or tool lets an agent do, and revoke it.
3. Reproduce a successful attack as a failing test, then patch it.
4. Write a blameless red-team report that another team could act on.

04 GitHub skill

Fine-grained personal access tokens, least privilege, revoking a leaked token

1. Create a fine-grained token scoped to one repository and the minimum permissions the agent needs.
2. Give the attackers an over-privileged token on the sandbox fork and watch what the agent can do with it.
3. Practise revocation: revoke a token, confirm the agent's next call fails, rotate.
4. Review the audit log afterwards to see what the token touched.

05 Class activity · Red team · blue team

90 min · Paired teams · disposable VMs

1. 0–10: Rules of engagement, target modules assigned, VMs handed out.
2. 10–70: Red phase. Try to change a grade, clear fees without paying, read another student's record, or poison the module's agent.
3. 70–80: Disclosure. Hand your target a written list of what worked.
4. 80–90: Blue phase starts. Reproduce each finding and open a fix PR.

Output. Attack log (red) and fix PRs with regression tests (blue).

THINK · PAIR · SHARE (2 / 4 / 6 MIN)

Which attack surprised you most, and which control would have stopped it at the earliest point in the loop?

05 Studio lab · Red-team / blue-team on another team's module

1. Confirm rules of engagement are signed and the VM is isolated. Start the clock.
2. Attack your target module: change a grade, clear fees without paying, read another student's record, poison its agent.
3. Defend your own: watch the logs and the audit chain, pull the kill switch if needed, note every anomaly.
4. Debrief together. Attackers reveal all attempts, successful or not.
5. Reproduce each successful attack as a failing test; patch; prove it with a passing test.
6. Write the red-team report and the honest journal entry.

Autonomy L2 for attack tooling inside the sandbox only; L1 for every patch. Nothing leaves the VM.
Budget Suggested ceiling: 300k tokens. Attacks are cheap; proofs are not.

06 · ASSIGNMENT A8

Red-team report and patches

DUE
Before the Week 9 seminar

Turn what broke into tests, so it can never break the same way again.

- [ ] Red-team report: attacks attempted, which succeeded, reproduction steps.
- [ ] Fix PRs for every successful attack on your system, each with a regression test and linked security issue.
- [ ] A private GitHub security advisory draft for your most serious finding.
- [ ] Journal entry 8: the honest one.

Submit on GitHub. Team PRs plus the report in docs/redteam/; individual attack credit claimed in the tracker issue.

Criterion	Weight
Attacks: creativity and evidence	30%
Patches with regression tests	30%
Advisory and disclosure quality	20%
Journal honesty	20%

Journal prompt. *Which attack on your system worked, how long before you noticed, and what would have made it visible sooner?*

Required reading A documented real-world indirect prompt-injection incident, selected by the instructor from the current year.
Optional The Model Context Protocol specification: the authorization section.

WEEK 09

OPERATE AND OWN · SYSTEMS ANALYSIS AND AGENTIC PRACTICE

Agent-Native Architecture

Designing systems that agents build and operate: legible repositories, narrow tool APIs, typed boundaries, ADRs as memory. Portability across harnesses.

01 Outcomes

1. Score a repository against the agent-readiness checks and raise the score with targeted refactors.
2. Design tool APIs narrow enough that misuse is hard and typed boundaries that make agent errors visible.
3. Separate the standard parts of your configuration (AGENTS.md, MCP) from the proprietary parts, and port it.
4. Use ADRs as the memory that outlives any harness.

04 GitHub skill

Repository hygiene, templates, Codespaces as a portable environment

1. Add issue and PR templates that carry the autonomy level and transcript link for any harness.
2. Define a devcontainer so the repository builds identically in Codespaces and locally; the second harness runs in it.
3. Record the agent-readiness score before and after in the README.
4. Open the port as its own PR, with the breakage log as the description.

05 Class activity · Readiness audit swap

25 min · Teams · peer audit

1. Swap modules with another team.
2. Run the 8-check agent-readiness playbook on their module and score each check 0–2.
3. Open a fresh agent session with no extra prompting: can it explain the module's events and run its tests?
4. File the three highest-value fixes as issues for their module.

Output. A readiness scorecard issue on the other team's repo; three fix issues.

THINK · PAIR · SHARE (2 / 4 / 6 MIN)

When you ported to the second harness, what broke? Was it the standard or the vendor-specific layer, and what does that tell you about lock-in?

05 Studio lab · Integrate the ERP through events, then make it agent-ready

1. Publish and consume your module's events exactly as docs/integration-contracts.md defines them.
2. Write an integration test for one cross-module journey that touches your module (e.g. admitted, fees cleared, registered).
3. Score your module against the 8 agent-readiness checks; refactor the two lowest areas.
4. Port your harness configuration to the secondary harness; have it build one small feature from your spec.
5. Log everything that broke and how you fixed or accepted it.
6. Open the PR with the integration test, before/after scores and the port log.

Autonomy L1 throughout. The second harness gets the same level and budget as the first; compare their transcripts.
Budget Suggested ceiling: 250k tokens across both harnesses.

06 · ASSIGNMENT A9

Integration and agent-readiness

Make your module part of one ERP, and easy for any agent to work on.

DUE
Before the Week 10 seminar

- [ ] Publish and consume your events exactly as docs/integration-contracts.md defines them.
- [ ] One integration test of a cross-module journey through your module.
- [ ] Readiness score before and after, with the refactor PRs between.
- [ ] Port your harness configuration and build one small feature with the second harness.
- [ ] Port report: what was standard, what was vendor-specific, what broke.

Submit on GitHub. Team PRs plus docs/port-report.md.

Criterion	Weight
Readiness improvement	30%
Second harness builds from the spec	25%
Port report insight	20%
Codespaces devcontainer	15%
Journal	10%

Journal prompt. *What did the second harness do differently with the same spec, and which difference was the configuration's fault?*

Required reading Configuring agentic AI coding tools: an exploratory study (arXiv 2602.14690, 2026).

Optional Context engineering for AI agents (metacto, 2026); How to make your repo agent-ready in 8 checks (roborhythms, 2026).

WEEK 10

OPERATE AND OWN · AGENTIC PRACTICE

Running Agents in Production

Cost and latency, model routing, autonomy ladders, human gates, scheduled and unattended agents, observability and incident response.

01 Outcomes

1. Route work between a cheap model and a frontier model and measure the difference in outcome and cost.
2. Design an L3 agent with a budget, a scope, a kill switch, a log and a named reader of the log.
3. Build human gates into an automated workflow and know when they are too many or too few.
4. Run a tabletop incident and produce a runbook from it.

04 GitHub skill

Scheduled Actions, environments with required reviewers, workflow cancellation

1. Write a scheduled workflow (cron) that runs the agent on a bounded task, e.g. triaging new issues or refreshing a dashboard.
2. Put its deploy step behind an environment with a required reviewer: the human gate.
3. Set a job timeout and a cost cap; document cancel-and-disable as the kill switch in the runbook.
4. Upload the transcript and cost as workflow artifacts; the Operator reads them daily.

05 Class activity · Tabletop incident 30 min · Teams of 4 · roles

1. Roles: incident commander, investigator, communicator, scribe.
2. Scenario cards arrive every 5 minutes: wrong fee notices to all students, the Bursar on the phone, a parent on radio, a cost alert.
3. Decide: stop, contain, investigate, communicate. The scribe keeps a timeline from the audit log.
4. Write a five-line blameless postmortem: what happened, why, what changes.

Output. Timeline and postmortem, committed as docs/incidents/tabletop-01.md.

THINK · PAIR · SHARE (2 / 4 / 6 MIN)

When does cheap-model-first cost more than it saves? Use your own experiment's numbers.

05 Studio lab · Run an ERP agent in production: gated, budgeted, observable

1. Run the cheap-model-first protocol on one real module task; fill the cost table.
2. Write a scheduled agent for your module (e.g. nightly fee-clearance reconciliation, unreleased-results check) with timeout, cost cap and approval gate.
3. Trigger it manually; approve the gate; read the artifacts and the audit log.
4. Tabletop: the scheduled agent emailed fee-defaulter notices to every student. Respond in real time.
5. Write the incident runbook from what you did and what you wish you had done.
6. Open the PR with the cost report, the workflow and the runbook.

Autonomy L3 for the scheduled agent, inside the gates you built. L1 for everything else.
Budget Suggested ceiling: 350k tokens, including both runs of the experiment.

06 · ASSIGNMENT A10

Production readiness

Run an agent unattended and be able to stop it, explain it and pay for it.

DUE
Before the Week 11 seminar

- [ ] Cost report: the same module task with cheap-model-first and frontier-only, compared on cost and outcome.
- [ ] A scheduled agent for your module as a GitHub Actions workflow, gated by an environment approval.
- [ ] A documented kill switch and token budget for that agent.
- [ ] An incident runbook for your module.
- [ ] The tabletop postmortem.

Submit on GitHub. Team PRs; the scheduled workflow visible in the Actions tab with at least one gated run.

Criterion	Weight
Scheduled agent with gates and kill switch	35%
Cost experiment rigour	25%
Runbook and postmortem	25%
Journal	15%

Journal prompt. *What would you need to see in the log to pull the kill switch, and who looks at the log on a Sunday?*

Required reading Allspaw, Blameless Post-Mortems and a Just Culture. Read before the tabletop.

Optional The SRE book: chapters on monitoring distributed systems and incident response.

WEEK 11

OPERATE AND OWN · SYSTEMS ANALYSIS AND SECURITY

Governance and the Changed Profession

Accountability for agent-built code, IP and licensing, data protection under the NDPA, audit trails, and IT project management when velocity is no longer the bottleneck.

01 Outcomes

1. Write a governance policy that says who is accountable for agent-built code and how that accountability is evidenced.
2. Apply the NDPA principles and controller obligations to the capstone's data flows.
3. Choose a licence and explain the IP position of agent-generated code in it.
4. Assemble a viva pack in which every claim links to a commit, a transcript or a test.

04 GitHub skill

Repository rulesets, audit log, licence, SECURITY.md and GOVERNANCE.md

1. Convert branch protection into a repository ruleset and export it as evidence.
2. Pull the organisation audit log for your repository; attach the relevant entries to the viva pack.
3. Choose a LICENSE and state in GOVERNANCE.md how agent-generated contributions are treated under it.
4. Link GOVERNANCE.md, SECURITY.md, THREATMODEL.md and the runbook from the README.

05 Class activity · Policy courtroom 30 min · Defence and prosecution teams

1. Each team brings its module's section of GOVERNANCE.md.
2. The prosecuting team draws a scenario: an agent changed a result after Senate approval; a transcript leaked another student's grades; agent-written code copied from a copyleft project.
3. Prosecution has 4 minutes to show the policy fails; defence has 4 minutes to show it holds.
4. The class rules. Losing clauses are rewritten before the session ends.

Output. Revised GOVERNANCE.md clauses in a PR, linked to the scenario that broke them.

THINK · PAIR · SHARE (2 / 4 / 6 MIN)

What does IT project management look like when velocity is no longer the bottleneck? What do you measure instead?

05 Studio lab · Govern the ERP: data protection, results integrity, viva prep

1. Write your module's section of GOVERNANCE.md: who may approve what, and what is never delegated to an agent.
2. Map NDPA principles to your module's personal data: lawful basis, retention, who can see it; file gaps as issues.
3. Record the IP position of agent-generated code and your licence decision in an ADR.
4. Assemble the viva pack: one page of claims, each linking to evidence in the repository.
5. Run the examiner agent against your module; answer its questions aloud, timed.
6. Fix whatever evidence you could not find in two minutes.

Autonomy L1 for drafts; L0 for the accountability statements and the refused-autonomy entry.
Budget Suggested ceiling: 200k tokens, mostly the examiner agent's reading.

06 · ASSIGNMENT A11

Governance and viva pack

DUE
Before the Week 12 session

Show who is accountable for what, and prepare to defend it.

- [ ] GOVERNANCE.md: accountability, data protection, licensing, audit trail, autonomy policy.
- [ ] CODEOWNERS covering security-sensitive paths.
- [ ] An automated licence check in CI.
- [ ] Viva pack: index of evidence, by decision.
- [ ] Dry-run viva with the examiner agent; transcript committed.

Submit on GitHub. Team PR; viva pack linked from the README.

Criterion	Weight
Governance completeness and realism	35%
CODEOWNERS and licence check	20%
Viva pack evidence index	25%
Dry-run reflection	20%

Journal prompt. *What did the examiner agent ask that you could not answer from the repository, and what did you add?*

Required reading Nigeria Data Protection Act 2023: the principles and controller obligations.

Optional Zuboff or Winner on technology and accountability, instructor's choice.

OPERATE AND OWN · ALL THREE STRANDS ASSESSED

WEEK 12

Capstone Demonstrations and Adversarial Viva

Each team demos, then faces twenty minutes of questioning from the instructor and an examiner agent that has read every transcript. Then the course asks what the agent should never have been allowed to do.

01 Outcomes

1. Demonstrate a working, agent-built information system from its tagged release.
2. Defend decisions, evidence and honesty in an adversarial viva, individually and as a team.
3. Locate any piece of evidence in your own repository within two minutes.
4. Articulate, from your journals, what you would and would not delegate again.

04 GitHub skill

Release tags, GitHub Releases, the repository as a portfolio

1. Tag the release candidate; the viva runs against that commit only.
2. Publish a GitHub Release with notes that link the spec, threat model, eval results, red-team report and governance policy.
3. Make the repository's README a one-page account of the system for a future employer or client.
4. Keep the repository: it is your portfolio and the account is yours.

05 Class activity · Demo, viva, retrospective

120 min · Whole class · scored

1. Class: one synthetic student's journey across every module, from admission to transcript.
2. Each team: 10-minute demo of its module from the tagged release, then a 20-minute adversarial viva.
3. The audience scores each demo against the rubric on the scoring sheet.
4. Closing retrospective: what should the agent never have been allowed to do in a university ERP?

Output. Viva scores, audience scores and the class retrospective board.

THINK · PAIR · SHARE (3 / 5 / 10 MIN)

Retrospective: looking back over twelve weeks of transcripts, what should the agent never have been allowed to do?

05 Studio lab · Demo the integrated ERP, defend your module, look back

1. Verify the tagged release builds clean from a fresh clone and seeds without errors.
2. Class demo: one synthetic student's journey across modules, from admission to transcript.
3. Team demo: your module from the tag, including its scheduled agent's last run and the audit trail.
4. Viva: answer from the repository, not from memory. Open the evidence as you speak.
5. Retrospective: reread your journal from week 1 and write the final entry.
6. Cohort discussion: what should the agent never have been allowed to do in a university ERP?

Autonomy L0. Today the humans do the talking.
Budget None for the viva. The examiner agent's reading was budgeted in week 11.

06 · ASSIGNMENT A12

Release, viva and portfolio

DUE
End of Week 12

Ship, defend, and leave with a portfolio that is yours.

- [ ] Tag v1.0 and publish a GitHub Release with notes.
- [ ] Pass the adversarial viva.
- [ ] Final journal entry: reread week 1 and write what changed.
- [ ] Pin the capstone on your GitHub profile with a README that explains your role and evidence.

Submit on GitHub. GitHub Release on the team repo; profile README on your own account.

Criterion	Weight
Release quality	25%
Viva: decisions, evidence, honesty	50%
Retrospective journal	15%
Portfolio README	10%

Journal prompt. *Reread week 1. What did you believe about agents then that you no longer believe, and what evidence changed it?*

Required reading Your own engineering journal, weeks 1 to 11, reread in one sitting.
Optional Brooks again, now with twelve weeks of evidence.